PROJECT REPORT

UCS310 – Database Management Systems

Green Campus Resource Optimization System

Submitted by:
Ayush Singh Sikarwar (Roll No: 1024030271)
Rishit Parnami (Roll No: 1024030270)

B.Tech – 2nd Year | Batch: 2025–2026
Lab Instructor: Mr. Deepak Sharma

Department of Computer Science & Engineering
Thapar Institute of Engineering and Technology, Patiala
January – June 2026

1. Introduction

Sustainable development is a major concern in modern educational institutions due to rapidly increasing consumption of natural resources such as electricity and water, along with challenges in proper waste management. College campuses consist of multiple buildings — academic blocks, hostels, libraries, and administrative offices — that collectively consume enormous quantities of resources daily.

The Green Campus Resource Optimization System is a database-driven solution designed to monitor, manage, and optimize campus resource usage. The system uses a centralized DBMS to store and analyze consumption data, enabling administrators to make informed, data-driven decisions toward long-term sustainability.

A Database Management System is preferred over a file-based approach because it eliminates data redundancy, ensures consistency across concurrent operations, improves security, and enables automation through SQL and PL/SQL constructs.

2. Problem Statement

Most campuses today lack a centralized system to monitor resource usage effectively. The existing manual or fragmented systems result in excessive wastage, inaccurate records, and

a complete absence of accountability. There is no automated mechanism to detect overconsumption in real time or to generate periodic sustainability reports.

This project addresses these limitations by implementing a fully structured database backend that tracks consumption across buildings, automatically generates sustainability alerts when thresholds are breached, and supports maintenance request management — all without requiring frontend development.

3. Objectives of the Project

- Design a complete Entity–Relationship (ER) model for campus resource management.
- Convert the ER model into normalized relational tables.
- Normalize the database up to Third Normal Form (3NF) to eliminate redundancy.
- Implement the database using structured SQL (DDL and DML).
- Use PL/SQL constructs — procedures, functions, triggers, and cursors — for automation.
- Ensure strict data consistency using CHECK constraints, foreign keys, and transactions.
- Provide a meaningful set of analytical queries and views for administrative reporting.

4. Scope of the Project

The project is strictly a backend database implementation; no frontend or user interface is developed. The scope covers the following functional areas:

- Resources tracked: Electricity (kWh), Water (Litres), and Waste (kg).
- User type: Admin (campus staff responsible for monitoring and resolving alerts).
- Modules: Resource consumption tracking, sustainability alert generation, maintenance request management, monthly reporting, and audit logging.

The system manages seven campus buildings across different types (Academic, Hostel, Library, Administrative, Laboratory) and stores all operational data in a single normalized MySQL database.

5. Proposed System Description

The system stores data related to campus buildings and their periodic resource consumption. Each building's usage of electricity, water, and waste is logged in the Consumption_Record table. If any consumption entry exceeds the pre-configured threshold for that resource, a trigger automatically fires and inserts a Sustainability_Alert record — ensuring no overconsumption event is ever missed.

Maintenance requests for resource-related issues (such as leaking pipes or malfunctioning AC units) are tracked in a separate table with priority levels and status workflow (Pending → In Progress → Resolved). Every status change is captured in an Audit_Log table via a dedicated trigger, providing a complete history of operational actions.

Stored procedures generate monthly resource reports, summarize building-level statistics, and allow batch acknowledgment of alerts. User-defined functions compute carbon footprints,

total consumption per building, and outstanding alert counts. Cursors are used to iterate over all over-threshold records in a structured, procedural manner.

6. Database Design

6.1 Entity–Relationship Description

The database comprises the following entities and their key relationships:

Entity	Primary Key	Key Attributes
Building	Building_ID	Building_Name, Building_Type, Location, Capacity, Established_Year
Resource	Resource_ID	Resource_Name, Unit, Threshold_Limit, Cost_Per_Unit
Consumption_Record	Record_ID	Building_ID (FK), Resource_ID (FK), Consumption_Amount, Record_Date
Maintenance_Request	Request_ID	Building_ID (FK), Issue_Description, Status, Priority, Request_Date
Sustainability_Alert	Alert_ID	Building_ID (FK), Resource_ID (FK), Alert_Message, Alert_Date, Severity, Acknowledged
Audit_Log	Log_ID	Table_Name, Operation, Record_Ref_ID, Changed_By, Changed_At, Details

Key relationships: One Building can have many Consumption Records; one Resource can be consumed by many Buildings; Alerts are automatically generated from Consumption Records; Maintenance Requests are linked to a Building.

6.2 Relational Schema

```
BUILDING(Building_ID PK, Building_Name, Building_Type, Location, Capacity,
Established_Year)
RESOURCE(Resource_ID PK, Resource_Name, Unit, Threshold_Limit, Cost_Per_Unit)
CONSUMPTION_RECORD(Record_ID PK, Building_ID FK, Resource_ID FK,
                   Consumption_Amount, Record_Date, Recorded_By)
MAINTENANCE_REQUEST(Request_ID PK, Building_ID FK, Issue_Description,
                    Status, Priority, Request_Date, Resolved_Date)
SUSTAINABILITY_ALERT(Alert_ID PK, Building_ID FK, Resource_ID FK,
                     Record_ID FK, Alert_Message, Alert_Date, Severity,
Acknowledged)
AUDIT_LOG(Log_ID PK, Table_Name, Operation, Record_Ref_ID, Changed_By, Changed_At,
Details)
```

7. Normalization

The database was designed and verified to comply with Third Normal Form (3NF) through the following steps:

- First Normal Form (1NF): All attributes hold atomic, indivisible values. No repeating groups exist. Each table has a well-defined primary key.
- Second Normal Form (2NF): All non-key attributes in tables with composite keys (e.g., Consumption_Record uses Building_ID + Resource_ID + Record_Date for its unique constraint) are fully functionally dependent on the entire key, not a partial subset.
- Third Normal Form (3NF): There are no transitive dependencies — no non-key attribute depends on another non-key attribute. Resource threshold and cost data are stored in the Resource table, not duplicated in Consumption_Record.

The final database schema is fully in Third Normal Form (3NF), ensuring minimal redundancy and maximum data integrity.

8. Database Implementation

8.1 DDL – Table Creation

The following core tables were created with appropriate data types, CHECK constraints, and foreign key references:

```
CREATE TABLE Building (
    Building_ID   INT PRIMARY KEY AUTO_INCREMENT,
    Building_Name VARCHAR(100) NOT NULL,
    Building_Type VARCHAR(50)  NOT NULL,
    Location      VARCHAR(100),
    Capacity      INT CHECK (Capacity > 0),
    Established_Year YEAR,
    CONSTRAINT chk_building_type
        CHECK (Building_Type IN ('Academic','Hostel','Library',
                                 'Administrative','Laboratory'))
);

CREATE TABLE Resource (
    Resource_ID      INT PRIMARY KEY AUTO_INCREMENT,
    Resource_Name    VARCHAR(50) NOT NULL UNIQUE,
    Unit             VARCHAR(20) NOT NULL,
    Threshold_Limit  DECIMAL(10,2) NOT NULL DEFAULT 1000.00,
    Cost_Per_Unit    DECIMAL(8,4)  NOT NULL DEFAULT 0.00
);

CREATE TABLE Consumption_Record (
    Record_ID          INT PRIMARY KEY AUTO_INCREMENT,
    Building_ID        INT  NOT NULL,
    Resource_ID        INT  NOT NULL,
    Consumption_Amount DECIMAL(10,2) NOT NULL CHECK (Consumption_Amount >= 0),
    Record_Date        DATE NOT NULL,
    FOREIGN KEY (Building_ID) REFERENCES Building(Building_ID) ON DELETE CASCADE,
    FOREIGN KEY (Resource_ID) REFERENCES Resource(Resource_ID) ON DELETE RESTRICT,
    UNIQUE (Building_ID, Resource_ID, Record_Date)
);
```

8.2 Sample Data Inserted

Seven campus buildings and three resources were seeded into the database, along with 15 consumption records spanning January to April 2025:

Building ID	Building Name	Type	Location	Capacity
1	Academic Block A	Academic	North Campus	500
2	Hostel Block B	Hostel	East Campus	300
3	Central Library	Library	Main Campus	200
4	Admin Office	Administrative	Main Campus	100
5	Science Lab	Laboratory	South Campus	150
6	Academic Block C	Academic	West Campus	450
7	Hostel Block D	Hostel	West Campus	350

Resource	Unit	Threshold Limit	Cost per Unit (₹)
Electricity	kWh	1000.00	7.50
Water	Litres	1000.00	0.03
Waste	kg	500.00	2.00

9. SQL Queries

Q1 – Full Consumption Detail (via View)

```
SELECT * FROM vw_consumption_detail ORDER BY Record_Date DESC;

-- Returns: Record_ID, Building_Name, Building_Type, Resource_Name,
--          Unit, Consumption_Amount, Threshold_Limit, Status, Record_Date
```

Q2 – Active Sustainability Alerts

```
SELECT * FROM vw_alert_summary WHERE Acknowledged = 'No';

-- Returns: Alert_ID, Building_Name, Resource_Name,
--          Alert_Message, Severity, Alert_Date, Acknowledged
```

Q3 – Pending Maintenance Requests

```
SELECT * FROM vw_pending_maintenance;

-- Returns: Request_ID, Building_Name, Issue_Description,
--          Status, Priority, Request_Date, Days_Open
```

Q4 – Carbon Footprint & Open Alerts per Building

```
SELECT b.Building_Name,
       fn_total_consumption(b.Building_ID, 1)  AS Total_kWh,
       fn_carbon_footprint(
```

```
           fn_total_consumption(b.Building_ID, 1)) AS Carbon_kg,
       fn_alert_count(b.Building_ID)           AS Open_Alerts
FROM Building b
ORDER BY Carbon_kg DESC;
```

Q5 – Monthly Resource Consumption Summary

```
SELECT MONTHNAME(Record_Date) AS Month,
       r.Resource_Name,
       SUM(Consumption_Amount) AS Total,
       r.Unit
FROM Consumption_Record cr
JOIN Resource r ON cr.Resource_ID = r.Resource_ID
GROUP BY MONTH(Record_Date), MONTHNAME(Record_Date),
         r.Resource_ID, r.Resource_Name, r.Unit
ORDER BY MONTH(Record_Date), r.Resource_ID;
```

Q6 – Top 3 Buildings by Total Resource Consumption

```
SELECT b.Building_Name,
       SUM(cr.Consumption_Amount) AS Grand_Total
FROM Consumption_Record cr
JOIN Building b ON cr.Building_ID = b.Building_ID
GROUP BY b.Building_ID, b.Building_Name
ORDER BY Grand_Total DESC
LIMIT 3;
```

Q7 – Buildings with No Alerts (Subquery)

```
SELECT b.Building_Name
FROM Building b
WHERE b.Building_ID NOT IN (
    SELECT DISTINCT Building_ID FROM Sustainability_Alert
);
```

Q8 – Estimated Cost per Building per Resource

```
SELECT b.Building_Name, r.Resource_Name,
       SUM(cr.Consumption_Amount)  AS Total_Units,
       r.Unit, r.Cost_Per_Unit,
       ROUND(SUM(cr.Consumption_Amount) * r.Cost_Per_Unit, 2)
           AS Estimated_Cost_INR
FROM Consumption_Record cr
JOIN Building b ON cr.Building_ID = b.Building_ID
JOIN Resource  r ON cr.Resource_ID  = r.Resource_ID
GROUP BY b.Building_ID, b.Building_Name,
         r.Resource_ID, r.Resource_Name, r.Unit, r.Cost_Per_Unit
ORDER BY Estimated_Cost_INR DESC;
```

10. PL/SQL Components

10.1 Trigger – Overconsumption Alert (trg_overconsumption_alert)

Fires AFTER every INSERT on Consumption_Record. If the new value exceeds the resource threshold, it automatically inserts a Sustainability_Alert with a descriptive message.

```
CREATE TRIGGER trg_overconsumption_alert
```

```
AFTER INSERT ON Consumption_Record
FOR EACH ROW
BEGIN
    DECLARE v_threshold DECIMAL(10,2);
    DECLARE v_resource  VARCHAR(50);
    DECLARE v_building  VARCHAR(100);
    DECLARE v_msg       VARCHAR(300);

    SELECT Threshold_Limit, Resource_Name
    INTO v_threshold, v_resource
    FROM Resource WHERE Resource_ID = NEW.Resource_ID;

    SELECT Building_Name INTO v_building
    FROM Building WHERE Building_ID = NEW.Building_ID;

    IF NEW.Consumption_Amount > v_threshold THEN
        SET v_msg = CONCAT(v_resource, ' overconsumption detected in ',
                           v_building, '. Recorded: ',
                           NEW.Consumption_Amount);
        INSERT INTO Sustainability_Alert
            (Building_ID, Resource_ID, Record_ID,
             Alert_Message, Alert_Date, Severity)
        VALUES (NEW.Building_ID, NEW.Resource_ID, NEW.Record_ID,
                v_msg, NEW.Record_Date, 'High');
    END IF;
END$$
```

10.2 Trigger – Audit Log on Maintenance Status Change (trg_log_status_change)

Fires AFTER UPDATE on Maintenance_Request. If the Status column changes, an Audit_Log entry is created recording the old and new values.

```
CREATE TRIGGER trg_log_status_change
AFTER UPDATE ON Maintenance_Request
FOR EACH ROW
BEGIN
    IF OLD.Status <> NEW.Status THEN
        INSERT INTO Audit_Log
            (Table_Name, Operation, Record_Ref_ID, Details)
        VALUES ('Maintenance_Request', 'UPDATE', NEW.Request_ID,
                CONCAT('Status changed from ', OLD.Status,
                       ' to ', NEW.Status));
    END IF;
END$$
```

10.3 Stored Procedure – Monthly Report (sp_monthly_report)

Accepts a month and year as input and outputs total consumption per resource for that period, alongside each resource's threshold.

```
CREATE PROCEDURE sp_monthly_report(IN p_month INT, IN p_year INT)
BEGIN
    SELECT r.Resource_Name, r.Unit,
           SUM(cr.Consumption_Amount) AS Total_Consumption,
           r.Threshold_Limit
    FROM Consumption_Record cr
    JOIN Resource r ON cr.Resource_ID = r.Resource_ID
    WHERE MONTH(cr.Record_Date) = p_month
      AND YEAR(cr.Record_Date)  = p_year
    GROUP BY r.Resource_ID, r.Resource_Name, r.Unit, r.Threshold_Limit;
END$$
```

```
-- Usage:
CALL sp_monthly_report(2, 2025);
```

10.4 Stored Procedure – Building Summary (sp_building_summary)

Takes a Building_ID and prints a complete summary including total consumption per resource, associated costs, and number of open alerts.

```
CREATE PROCEDURE sp_building_summary(IN p_building_id INT)
BEGIN
    SELECT b.Building_Name, b.Building_Type,
           r.Resource_Name, r.Unit,
           SUM(cr.Consumption_Amount) AS Total_Consumed,
           ROUND(SUM(cr.Consumption_Amount) * r.Cost_Per_Unit, 2) AS Cost_INR
    FROM Consumption_Record cr
    JOIN Building b ON cr.Building_ID = b.Building_ID
    JOIN Resource  r ON cr.Resource_ID  = r.Resource_ID
    WHERE cr.Building_ID = p_building_id
    GROUP BY r.Resource_ID, r.Resource_Name, r.Unit, r.Cost_Per_Unit,
             b.Building_Name, b.Building_Type;
END$$

-- Usage:
CALL sp_building_summary(1);
```

10.5 Cursor Procedure – All Over-Threshold Records (sp_cursor_all_overconsumption)

Uses an explicit cursor to iterate row-by-row over every consumption record that exceeds its threshold, computing the excess percentage for each.

```
CREATE PROCEDURE sp_cursor_all_overconsumption()
BEGIN
    DECLARE v_done INT DEFAULT FALSE;
    DECLARE v_building VARCHAR(100);
    DECLARE v_resource VARCHAR(50);
    DECLARE v_amount   DECIMAL(10,2);
    DECLARE v_thresh   DECIMAL(10,2);

    DECLARE cur_over CURSOR FOR
        SELECT b.Building_Name, r.Resource_Name,
               cr.Consumption_Amount, r.Threshold_Limit
        FROM Consumption_Record cr
        JOIN Building b ON cr.Building_ID = b.Building_ID
        JOIN Resource  r ON cr.Resource_ID  = r.Resource_ID
        WHERE cr.Consumption_Amount > r.Threshold_Limit
        ORDER BY cr.Consumption_Amount DESC;

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = TRUE;
    OPEN cur_over;
    over_loop: LOOP
        FETCH cur_over INTO v_building, v_resource, v_amount, v_thresh;
        IF v_done THEN LEAVE over_loop; END IF;
        SELECT v_building AS Building, v_resource AS Resource,
               v_amount  AS Consumed,  v_thresh  AS Threshold,
               ROUND(((v_amount-v_thresh)/v_thresh)*100,1) AS Excess_Pct;
    END LOOP;
    CLOSE cur_over;
END$$
```

10.6 Functions

Three user-defined functions encapsulate frequently used calculations:

Function	Parameters	Description
fn_carbon_footprint()	p_consumption NUMBER	Returns CO_2 equivalent (consumption × 0.82)
fn_total_consumption()	p_building_id, p_resource_id	Returns total consumption for a building/resource pair
fn_alert_count()	p_building_id	Returns count of unacknowledged alerts for a building

```
-- Usage examples:
SELECT fn_carbon_footprint(1500) AS Carbon_1500kWh;
SELECT fn_total_consumption(1, 1) AS AcademicBlockA_Electricity_kWh;
SELECT fn_alert_count(1) AS OpenAlerts_AcademicBlockA;
```

11. Database Views

Three views were created to simplify repeated query patterns and provide a clean interface for administrative reporting:

View Name	Purpose
vw_consumption_detail	Joins Building, Resource, and Consumption_Record; adds a computed Status column ('OVER' or 'Normal') based on threshold comparison.
vw_alert_summary	Joins Sustainability_Alert with Building and Resource; converts the Acknowledged TINYINT to a readable 'Yes'/'No' label.
vw_pending_maintenance	Shows all non-resolved Maintenance Requests joined with Building name, including a Days_Open computed column; sorted by priority then date.

12. Transaction Management

The system uses explicit transactions to ensure atomicity when multiple related operations must succeed or fail together. The example below logs a new over-threshold consumption entry (which fires the alert trigger automatically) and simultaneously updates the related maintenance request status:

```
START TRANSACTION;

-- Step 1: Log new consumption (fires trg_overconsumption_alert automatically)
INSERT INTO Consumption_Record
    (Building_ID, Resource_ID, Consumption_Amount, Record_Date)
VALUES (1, 1, 1350.00, '2025-05-07');
```

```
-- Step 2: Update maintenance status (fires trg_log_status_change automatically)
UPDATE Maintenance_Request
SET    Status = 'In Progress'
WHERE  Building_ID = 1
  AND  Status      = 'Pending'
  AND  Priority    = 'High'
LIMIT 1;

COMMIT;

-- If any step fails, ROLLBACK would undo all changes.
```

13. Expected Outcomes

- Centralized, real-time monitoring of all campus resource consumption.
- Automatic generation of sustainability alerts when thresholds are exceeded, eliminating manual oversight.
- Complete maintenance request lifecycle tracking with full audit trail.
- Monthly and building-level reporting accessible with a single procedure call.
- Improved data consistency and integrity through normalization, constraints, and transactions.
- Support for sustainable campus development through data-driven decision making.

14. Tools & Technologies Used

Component	Details
DBMS Software	MySQL 8.x
Query Language	SQL (DDL, DML, DQL) and Stored Procedure language (PL/SQL-style MySQL)
Interface Tool	MySQL Workbench / CLI
Programming	JavaScript (Node.js) for report generation
Academic Year	January – June 2026

15. Conclusion

The Green Campus Resource Optimization System successfully demonstrates how a well-designed relational database can serve as the backbone for an institution-wide sustainability initiative. By centralizing consumption data, enforcing thresholds through automated triggers, and exposing actionable insights via views and stored procedures, the system transforms raw meter readings into a structured decision-support framework.

The project satisfies all stated objectives: the schema is normalized to 3NF, all key PL/SQL constructs (triggers, procedures, functions, and a cursor) are implemented and tested, and data integrity is maintained through foreign keys, CHECK constraints, and explicit transaction management.

Future enhancements could include a web-based frontend dashboard, IoT sensor integration for real-time data ingestion, and predictive analytics using historical consumption trends to forecast future overconsumption events before they occur.